

StellarIntel: Automated Satellite Telemetry Analysis AI Application

Learning Block 1 Report

Submitted by
Mahek Patel

G H Patel College of Engineering And Technology

Department of Computer Engineering
Academic Year: Pre-Final (2026-2027)

Guided by: Harsha Pedhadiya

Index

No.	Section Title	Coverage & Key Focus Areas
1	1. Introduction	Overview, Real-World Purpose, GenAI Role, Output Formats, Core Features
2	2. Problem Statement	Operational Difficulties, Inefficiencies, GenAI Suitability, Expected Outcomes
3	3. Objectives	Measurable Engineering Objectives for Telemetry, AI, RAG, UI, Evaluation & PDF
4	4. Project Scope	Covered Use Cases, Target User Personas, Included Features, Boundaries
5	5. Proposed System / Methodology	Dual-Engine Architecture, Multi-Stage Pipeline, Validation, RAG Retrieval
6	6. System Architecture / Workflow	Component Interactions, Prompt Construction Layer, Spring AI Integration
7	7. Implementation	Tech Stack, Backend & Frontend Development, Spring AI RAG Setup, Code Snip
8	8. User Interface / Application Walkthrough	Dashboard Tour, Preset Profiles, Instruments Bus, RAG Context, Report Generato
9	9. Challenges and Limitations	Hallucination Prevention, API Quotas & Fallbacks, Type Safety, PDF Layouts
10	10. Conclusion	Project Summary, GenAI Contribution, Key Architecture Takeaways, Learning Out
11	11. Future Scope	Kafka Streaming, Predictive Forecasting, Multimodal Imaging, Telecommands
12	12. References	IEEE / NASA Aerospace Handbooks, Spring AI Documentation, Academic Citation

1. Introduction

StellarIntel is an enterprise-grade satellite telemetry analysis application that bridges the gap between raw spacecraft telemetry data and explainable artificial intelligence. Developed using Java 21, Spring Boot 3.3.4, and Spring AI, the application autonomously ingests satellite telemetry, detects multi-subsystem anomalies via deterministic rule evaluation, retrieves grounded domain knowledge through an in-memory VectorStore RAG pipeline, and synthesizes structured, evidence-based diagnostics using modern Large Language Models.

1.1 Real-World Purpose & Problem Context

In satellite mission operations, ground controllers monitor critical telemetry downlinks during short line-of-sight passes. Subsystem failures—such as battery thermal runaway in the Thermal Control Subsystem (TCS), solar array under-voltage in the Electrical Power Subsystem (EPS), reaction wheel friction in Attitude Determination and Control (ADCS), or solar particle events—require immediate investigation. StellarIntel empowers Flight Directors and operations personnel with an automated, explainable decision-support system that eliminates the need for slow, error-prone manual lookups in paper binders.

1.2 Generative AI Role & Structured Output

Generative AI in StellarIntel serves as an evidence-grounded reasoning layer. By constraining the model to retrieved technical flight documentation, the system generates structured, type-safe JSON diagnostics containing Executive Summaries, Observed Spacecraft Conditions, Possible Contributing Factors, Affected Subsystems, and Recommended Operational Investigations.

1.3 Core Features

- **Telemetry Ingestion & Range Validation:** Validates sensor packets against strict aerospace boundaries.
- **Deterministic Anomaly Engine:** Evaluates mathematical ground truth to prevent false alarms or missed anomalies.
- **VectorStore RAG Retrieval:** Indexes 40 semantic chunks across 5 technical flight manuals for grounded context.
- **Autonomous Offline Fallback Engine:** Gracefully handles API credit exhaustion (HTTP 429) without downtime.
- **Automated PDF Mission Reporting:** Generates professional flight incident reports via OpenPDF streaming.

2. Problem Statement

Satellite monitoring is bottlenecked by **alert fatigue** and **fragmented flight manuals**. When complex orbital disturbances occur (e.g., solar flares or geomagnetic storms), dozens of sensor alarms fire simultaneously. Traditional telemetry systems rely solely on static binary threshold tripwires (e.g., 'Temp > 45°C'). They notify operators that a metric is out of bounds, but cannot explain *why* the failure occurred or correlate cross-subsystem effects (such as radiation causing bitflips in an ADCS microcontroller).

By fusing deterministic numerical ground truth with Retrieval-Augmented Generation, StellarIntel reduces root-cause triage time from 15–45 minutes of manual triage down to under 2 seconds of automated, verifiable synthesis, standardizing flight logging and mitigating catastrophic mission risks.

3. Objectives

- **Deployable Spring Boot 3 Architecture:** Implement an enterprise-grade Java 21 web application.
- **Deterministic Mathematical Ground Truth:** Implement deterministic anomaly isolation across 4 subsystem categories.

- **Semantic VectorStore RAG System:** Ingest 5 technical documentation manuals chunked into 40 vector embeddings.
- **Type-Safe GenAI Diagnostic Pipeline:** Integrate OpenAI GPT-4o-mini using Spring AI BeanOutputConverter.
- **Defensive Offline Fallback Engine:** Synthesize diagnostics deterministically when external API keys are unavailable.
- **Automated PDF Document Generator:** Build an in-memory OpenPDF service producing downloadable mission reports.
- **100% Test Passing Verification:** Achieve 55/55 passed unit, integration, and prompt-evaluation tests.

4. Project Scope

In-Scope: Ingestion of JSON telemetry packets (satelliteId, timestamp, batteryTemperatureCelsius, solarPanelVoltage, attitudeControlErrorX/Y, radiationExposureLevel); deterministic anomaly classification; semantic RAG retrieval; structured diagnostic generation; interactive space-themed web dashboard; and downloadable PDF mission reports.

Out-of-Scope: Direct RF satellite uplink telecommanding (read-only advisory system); multi-year historical telemetry databases; live radio frequency demodulation hardware.

5. Proposed System / Methodology

StellarIntel implements a multi-stage sequential pipeline that maintains strict separation between numerical truth and AI interpretation:

1. **Validation:** Telemetry is checked for syntax and realistic orbital bounds.
2. **Deterministic Evaluation:** Rule engine tags anomalies (NOMINAL, WARNING, CRITICAL).
3. **RAG Context Search:** VectorStore retrieves the top-4 relevant aerospace documentation excerpts.
4. **AI Synthesis:** GPT-4o-mini generates structured JSON diagnostics (or triggers local offline fallback).
5. **Presentation & Export:** Dashboard updates live instruments and enables 1-click PDF mission report export.

6. System Architecture / Workflow

Pipeline Stage	Software Component	Key Responsibilities & Data Hand-off
1. Ingestion	TelemetryApiController	Accepts JSON payload; executes Jakarta Bean Validation rules.
2. Rule Engine	AnomalyDetectionEngine	Calculates numerical threshold deviations; sets deterministic ground truth.
3. Knowledge RAG	KnowledgeRetrievalService	Formulates dynamic query; queries SimpleVectorStore (40 chunks).
4. GenAI Synthesis	GenAiDiagnosticService	Executes Spring AI ChatModel with BeanOutputConverter or Offline Fallback.
5. Report Generation	DiagnosticReportService	Builds OpenPDF Document stream with running header and footers.

7. Implementation

The application is implemented in Java 21 LTS using Spring Boot 3.3.4, Spring AI 1.0.0-M3, and OpenPDF 2.0.3. The frontend is built with clean semantic HTML5, Vanilla CSS3 utilizing a custom glassmorphism design system, and modern ES6+ JavaScript. Zero heavyweight UI frameworks were used, giving complete control over performance and responsiveness.

8. User Interface / Application Walkthrough

- **Mission Control Dashboard:** Dark-mode space UI with live telemetry synchronization and system clock.
- **Data Ingestion Panel:** Preset dropdown supporting 5 orbital profiles, JSON editor, and 2-row button layout.
- **Live Telemetry Instrument Bus:** Reactive meters for Battery Temp, Solar Voltage, ADCS X/Y, and Radiation.
- **Deterministic Anomaly Table:** Tabulates anomaly name, severity, measured value, threshold, and subsystem.
- **Retrieved Mission Knowledge:** Displays RAG document titles, topics, relevance scores, and technical excerpts.
- **GenAI Diagnostic Command Center:** Comprehensive root-cause analysis, contributing factors, and recovery checklist.
- **Automated PDF Mission Report:** Direct export button downloading professional reports with sanitized filenames.

9. Challenges and Limitations

Technical Challenges & Solutions:

- *Hallucination Mitigation:* Solved by separating deterministic Java rules from LLM reasoning and injecting RAG context.
- *API Quota Resilience:* Handled via an Autonomous Offline Fallback Engine that locally synthesizes diagnostics if OpenAI credits expire.
- *Type-Safe JSON:* Enforced using Spring AI BeanOutputConverter with secondary Jackson regex parsing.
- *PDF Document Generation:* Built with OpenPDF and a custom two-pass page event handler for 'Page X of Y' numbering.
- *Button Overflow:* Resolved by re-architecting the ingestion action buttons into a structured two-row responsive grid.

10. Conclusion

StellarIntel successfully establishes an operational paradigm where Generative AI and RAG augment flight operators rather than replacing them. By enforcing deterministic ground truth, grounding LLM inferences in authoritative documentation, and providing resilient offline fallbacks, the project demonstrates how enterprise Java and Spring AI can solve real-world aerospace operational bottlenecks.

11. Future Scope

- **Real-Time Kafka / MQTT Ingestion:** Connect live ground station downlinks for continuous telemetry analysis.
- **Predictive Time-Series ML:** Use BigQuery ML or LSTM networks to forecast thermal runaway 2 orbits in advance.
- **Multimodal Diagnostics:** Ingest star tracker optical camera images and radio frequency waterfall plots.
- **Automated Telecommand Generation:** Synthesize flight recovery scripts with cryptographic human verification.
- **Cloud Native Containerization:** Deploy containerized microservices on Google Cloud Run or Kubernetes.

12. References

1. VMware Tanzu / Pivotal. *Spring AI Reference Documentation (v1.0.0-M3)*, <https://docs.spring.io/spring-ai/reference/>
2. OpenAI. *GPT-4o and GPT-4o-mini Technical Documentation*, <https://platform.openai.com/docs/>
3. NASA. *Systems Engineering Handbook & ADCS Guidelines*, NASA SP-2016-6105.

4. European Space Agency (ESA). *Telemetry and Telecommand Packet Utilization Standard*, ECSS-E-ST-70-41C.
5. Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020.
6. LibrePDF. *OpenPDF: Open Source Java Library for PDF Generation*, <https://github.com/LibrePDF/OpenPDF>
7. Spring IO. *Spring Boot 3.3 Reference Manual*, <https://docs.spring.io/spring-boot/docs/3.3.4/reference/html/>
8. StellarIntel Project Repository, G H Patel College of Engineering And Technology, Pre-Final (2026-2027).